International Journal
Information Technology Applications (ITA)

Volume XX, Number X, Feb. 2026

DESIGN AND IMPLEMENTATION OF A WORKFLOW-ENABLED ENTERPRISE DOCUMENT MANAGEMENT SYSTEM WITH LEGISLATIVE COMPLIANCE

Juraj Suchán

Abstract: This paper presents the design and implementation of an enterprise document and records management system with integrated workflow support. The proposed solution addresses the need for structured management of registry records, including classification, retention policies, disposal procedures, and auditability, while maintaining extensibility and modern architectural principles. The system is based on a microservice-oriented architecture implemented in .NET, utilizing a relational database for structured metadata storage and distributed components for messaging and caching.

A dedicated workflow engine enables the definition and execution of configurable process models, including task, decision, and scripted nodes, allowing dynamic business logic execution and versioned workflow definitions. The data model reflects hierarchical classification structures and regulatory requirements related to records lifecycle management. The implementation emphasizes modularity, traceability, and compliance with records management principles [3, 5].

The resulting system demonstrates that enterprise-grade document and records management functionality can be combined with flexible workflow execution within a unified architecture, while preserving scalability and maintainability.

Keywords:
document management system, workflow engine, legislative compliance

ACM Computing Classification System:
Software and its engineering → Software creation and management → Software development techniques

Introduction

Enterprise document and records management systems (DMS/RMS) are fundamental components of contemporary information infrastructures. They ensure structured storage, controlled access, traceability, and lifecycle management of organizational documents. In regulated environments, such systems must additionally support compliance with legal and normative requirements related to classification schemes, retention periods, disposal procedures, and auditability.

While traditional document management platforms primarily address storage and retrieval functionality, advanced process coordination is often delegated to separate workflow or business process management systems. This architectural separation increases integration complexity, introduces data redundancy, and may result in inconsistencies in document lifecycle control. Consequently, organizations face challenges in maintaining a coherent link between records management principles and operational process execution. Existing standards such as ISO 15489 and MoReq2010 define conceptual and functional requirements for records management [3, 5], yet their practical integration with configurable workflow engines is frequently left to system-specific implementations.

This paper presents the design and implementation of a prototype enterprise document and records management system with integrated workflow support. The proposed solution adopts a unified architectural approach in which structured records modeling and configurable workflow execution are treated as mutually dependent components rather than isolated subsystems. The system is implemented using modern architectural principles, emphasizing modularity, scalability, and compliance-oriented data modeling.

The primary contribution of this work lies in: (i) the integration of a versioned and configurable workflow engine directly into a structured records management model, (ii) the alignment of workflow execution mechanisms with hierarchical classification structures and retention policies, and (iii) the architectural design of a modular system that supports auditability and extensibility without sacrificing maintainability.

The objective of this paper is to describe the architectural design, data model, workflow engine structure, and compliance mechanisms of the proposed solution, and to demonstrate how these elements form a coherent and extensible enterprise-ready system.

1 Background and Related Work

Document and records management systems have long been examined within the field of enterprise information systems and information governance. Normative frameworks such as ISO 15489 and MoReq2010 define conceptual and functional requirements for records lifecycle management, including classification schemes, retention schedules, disposal procedures, and auditability [3, 5]. These standards emphasize structured metadata models, hierarchical classification, and controlled disposition mechanisms to ensure long-term traceability and legal compliance.

From an architectural perspective, contemporary enterprise software increasingly adopts service-oriented or microservice-based approaches to enhance scalability, modularity, and independent component evolution. At the same time, workflow management systems and business process engines have evolved to support configurable process definitions, dynamic routing logic, and distributed task orchestration. Such systems frequently rely on graph-based process representations and versioned execution models to enable adaptability in changing business environments.

Despite these advancements, document management and workflow orchestration are commonly implemented as separate subsystems. In many enterprise solutions, workflow engines operate on top of document repositories through integration layers or API-based coupling. While this separation promotes loose coupling, it can also introduce inconsistencies in lifecycle state management, duplication of contextual data, and increased architectural complexity. The document lifecycle may be governed partly by records management rules and partly by external workflow states, leading to potential ambiguity in compliance-sensitive scenarios.

Several research contributions and industrial platforms propose tighter integration between content management and process execution. However, existing implementations often prioritize either robust document storage with compliance support or highly flexible workflow orchestration, rather than designing both aspects as inherently interdependent components of a single domain model.

The approach presented in this paper builds upon established records management principles while embedding a configurable workflow engine directly into the core data and execution model of the system. Instead of treating workflow as an external automation layer, workflow execution is aligned with classification structures, retention logic, and audit mechanisms. The contribution does not redefine existing standards, but demonstrates an architectural strategy for integrating compliance-oriented records modeling with versioned and extensible workflow execution within a unified enterprise system.

2 System Architecture

The proposed system follows a modular service-oriented architecture designed to ensure scalability, extensibility, and separation of concerns. The backend is implemented using the .NET 10 platform [6] and is structured into multiple logical components responsible for records management, workflow execution, authentication, and supporting infrastructure services.

Structured metadata related to records, classification hierarchies, workflow definitions, and execution states are stored in a relational database (Microsoft SQL Server). This choice enables strong consistency guarantees and transactional integrity for compliance-sensitive operations.

Binary document content is stored separately from structured metadata. Uploaded files are persisted in a document-oriented storage system (MongoDB), which is optimized for handling large binary objects and scalable file storage. This architectural separation between metadata and file content improves flexibility, supports horizontal scaling of storage components, and reduces performance bottlenecks in relational persistence layers.

Inter-service communication and asynchronous processing are supported by a message broker (RabbitMQ), which facilitates decoupled execution of background tasks and workflow-related events. A distributed caching layer (Redis) is employed to improve performance for frequently accessed metadata and configuration data.

Authentication and authorization are handled via an external identity provider (Keycloak), enabling standards-based authentication protocols and role-based access control. The system exposes its functionality through a REST-based API layer, allowing integration with external systems and frontend clients.

The user interface is implemented as a single-page application using React and TypeScript. A component-based UI framework is employed to ensure consistency and maintainability. This separation between backend services and frontend client supports independent evolution of presentation and business logic layers.

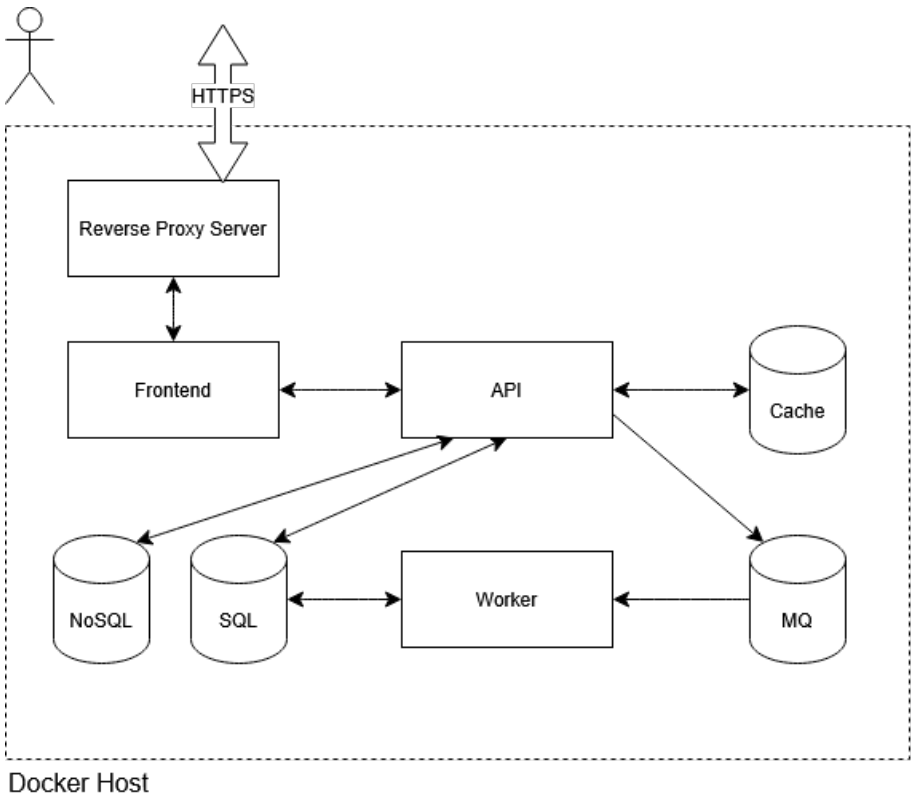

Figure 1: High-level system architecture of the proposed DMS solution.

For deployment, the system is containerized and supports Docker-based orchestration. The architecture is designed to be environment-agnostic, enabling both local development deployments and cloud-ready configurations. This infrastructure approach aligns with modern enterprise deployment

practices and ensures portability across different hosting environments.

The architectural decisions follow established enterprise design principles [4, 9] and align with modern service-oriented system patterns.

2.1 Configurable Metadata Model

In addition to core record attributes, the system supports configurable metadata bound to the evidence type (registry evidence category). Each evidence type defines a metadata schema consisting of user-defined fields that extend the base record model. These field definitions are automatically projected into record creation and update forms, allowing organizations to adapt the system to heterogeneous document agendas without modifying application code.

Metadata definitions are stored in normalized relational form. Each metadata field specifies a logical identifier, data type, validation constraints, optional enumeration values, and presentation-related parameters. This schema layer represents the authoritative definition of allowed metadata structure for a given evidence type.

At runtime, record-level metadata values are persisted as a serialized structured payload within the *RecordMetadata* entity. Instead of storing each metadata field as a separate relational column, the system stores a JSON-based dictionary containing key–value pairs corresponding to the defined metadata schema. Prior to persistence, this payload is validated against the relational metadata definition to ensure type consistency, mandatory field enforcement, and controlled enumeration values.

The frontend dynamically renders form controls based on the evidence-level metadata definition, while the backend enforces schema validation to prevent structural inconsistencies and schema drift. This hybrid design combines the flexibility of semi-structured runtime storage with the integrity guarantees of schema-driven validation.

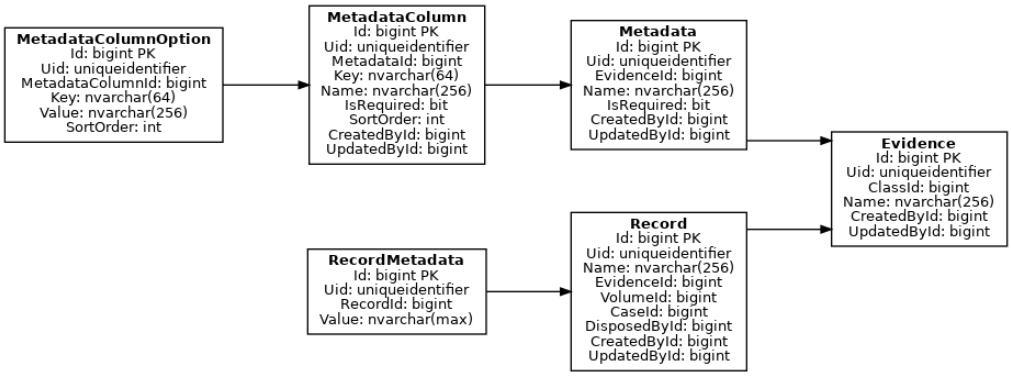

Figure 2: Evidence-bound metadata model and runtime metadata storage.

From a compliance perspective, evidence-bound metadata improves traceability and classification quality by ensuring that domain-specific attributes required by internal policies or legislation are consistently captured during record registration. By separating metadata definition (schema layer) from metadata instantiation (runtime layer), the system preserves reproducibility of record descriptions and supports auditability of historical record states. This approach aligns with records management principles defined in ISO 15489 and MoReq2010, which emphasize controlled metadata structures, lifecycle context, and traceable record evolution [3, 5].

The overall design follows a schema-driven persistence strategy, reducing implementation-level customization and enabling controlled evolution of metadata requirements through configuration rather than redeployment.

3 Workflow Engine Design

The workflow engine represents a central architectural component of the proposed system. Its purpose is to integrate process orchestration directly into the records management platform, thereby eliminating the need for external workflow subsystems and reducing integration complexity. The engine is designed to support configurability, version control, persistent execution state, and dynamic evaluation of process logic.

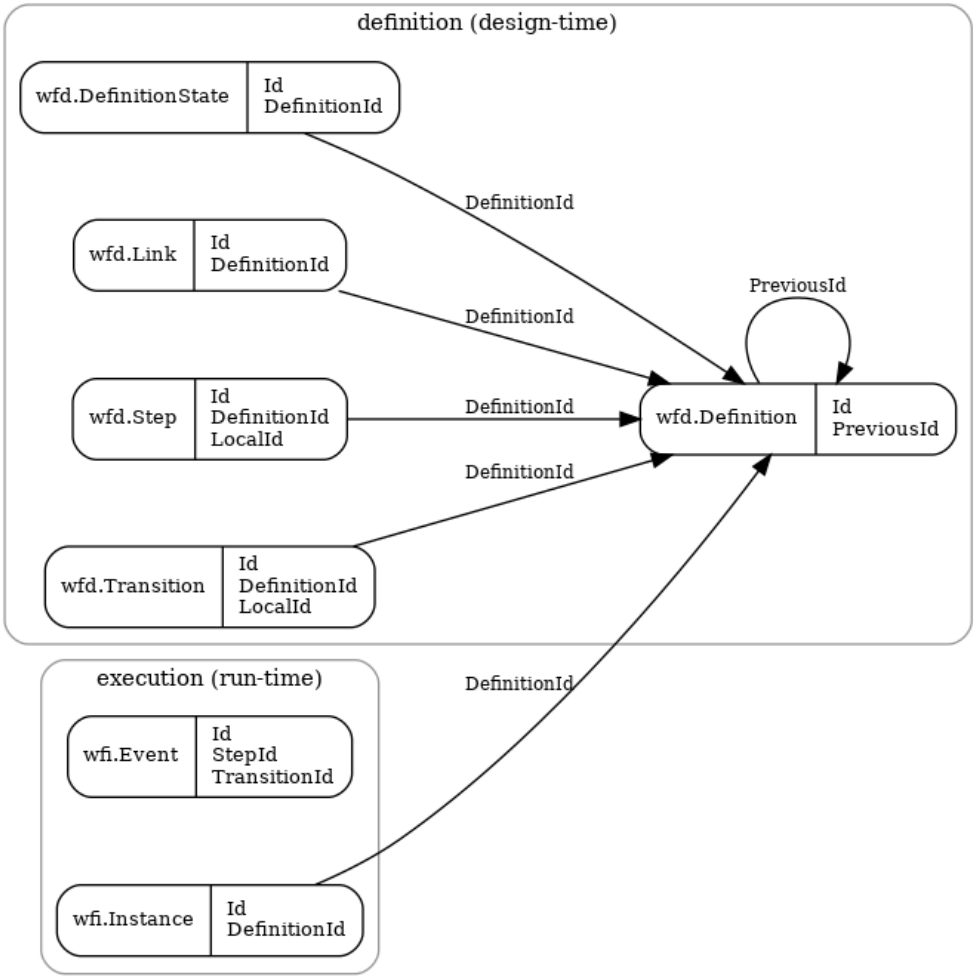

Figure 3: Workflow engine high level model.

Unlike traditional hard-coded business logic implementations, the workflow layer allows process models to be defined, modified, and versioned independently of application redeployment. This design enables organizations to adapt their document-related processes (e.g., approvals, registrations, reviews, or disposal procedures) without modifying the core application codebase.

3.1 Workflow Definition Model

Workflow processes are modeled as directed graphs consisting of nodes (steps) and transitions. The definition layer includes explicit support for versioning: each workflow definition is stored together with a snapshot of its structure, and newer versions maintain a reference to the previous version. This approach allows controlled evolution of workflow logic while preserving historical definitions for traceability and reproducibility.

The engine supports multiple node types, including start and end nodes, human task nodes, decision nodes, and script nodes.

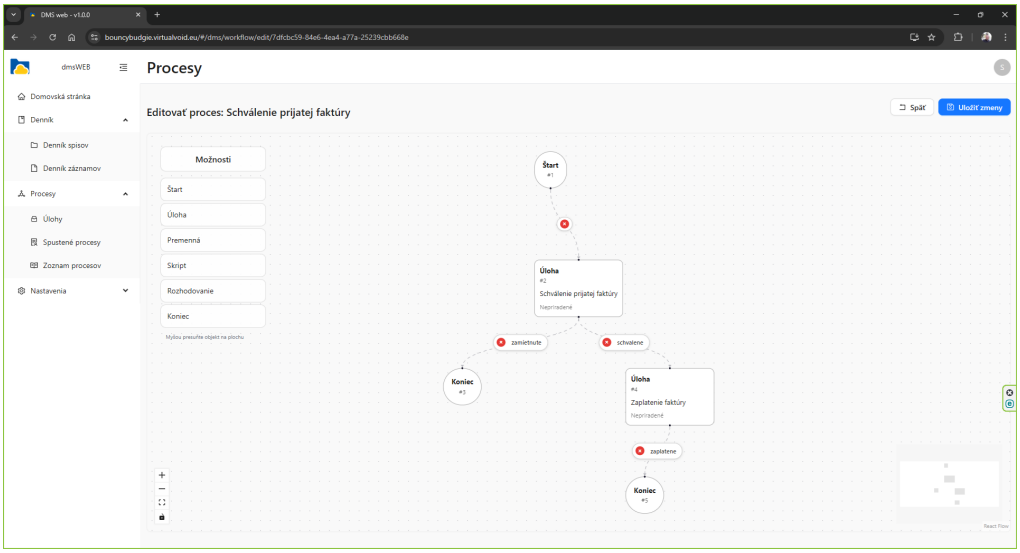

Figure 4: Example of a configurable workflow definition for invoice approval.

Transitions define the control flow between nodes and may include conditions or metadata used by the execution layer.

3.2 Execution Model

Workflow execution is represented by workflow instances that reference a specific version of a workflow definition. Each instance maintains persistent execution state, including the current node, completed steps, and contextual data required to evaluate transitions and decision logic. Persisting execution state enables fault tolerance and supports auditing of workflow progress over time.

Asynchronous execution and background processing are supported through message-based task dispatching, allowing workflow-related activities to be decoupled from synchronous API calls. This model improves scalability and helps ensure that long-running workflow steps do not block user interactions.

3.3 Script Nodes and Dynamic Logic

To enable flexible business logic without redeployment, the workflow engine provides script nodes. These nodes execute embedded scripts during runtime to evaluate conditions, compute values, or perform lightweight validation and routing logic. Script node outputs can influence transition selection and enable dynamic behavior in workflow graphs while keeping the core execution engine stable.

Overall, the workflow engine design balances configurability and maintainability by separating workflow structure (definitions) from runtime state (executions), while ensuring that versioned definitions remain available for auditing and consistent replay of historical workflow instances.

Workflow modeling principles are consistent with records lifecycle management standards

defined in ISO 15489 [5] and MoReq2010 requirements [3].

4 Legislative Compliance Model

The records management component of the system is explicitly designed to reflect legally grounded principles of structured classification, controlled retention, and auditable lifecycle management. Rather than treating compliance as an external constraint, regulatory requirements are embedded directly into the core domain model.

The data model incorporates hierarchical classification entities representing subject-based groupings of records. These entities correspond to registry plans and classification structures defined in archival practice, enabling systematic categorization and consistent assignment of documents to predefined groups. Such hierarchical modeling ensures semantic clarity, supports standardized numbering schemes, and enables structured aggregation of records across organizational units.

The core domain model distinguishes between evidence types, volumes, records, and case files. Evidence types represent abstract registry categories, while volumes and records represent their concrete temporal or contextual instantiations. This layered modeling approach separates classification semantics from operational instances, ensuring scalability of registry data while preserving logical continuity and traceability.

Retention and disposal attributes are modeled as explicit lifecycle properties associated with classification entities. Each classification group may define retention periods, disposal categories, and archival handling rules. Although full lifecycle automation (e.g., automatic disposal workflows) may evolve over time, the structural representation of retention metadata is embedded at the schema level, ensuring that lifecycle logic can be implemented without fundamental restructuring of the data model.

Auditability is ensured through persistent recording of creation, modification, and attribution metadata across compliance-sensitive entities. Timestamp fields and user identifiers are tightly integrated with the authentication subsystem, enabling accountability, traceability of actions, and reconstruction of record evolution. This design aligns with transparency requirements and supports inspection or audit scenarios.

By integrating hierarchical classification, lifecycle attributes, and audit mechanisms into the core domain model, the system establishes a structurally compliant foundation rather than relying solely on procedural enforcement.

The compliance model is derived from Slovak archival legislation, including Act No. 395/2002 Coll. on Archives and Registries [8] and Decree No. 628/2002 Coll. [7]. In addition, the system design considers EU-level regulatory frameworks such as the eIDAS Regulation [1], which governs trust services and electronic identification, and the General Data Protection Regulation (GDPR) [2], which introduces requirements related to personal data processing, accountability, and retention limitation.

5 Discussion

The presented architecture demonstrates that integrating records management and workflow orchestration within a unified domain model can reduce structural fragmentation commonly observed in enterprise information systems. In traditional environments, document storage, lifecycle management, and process execution are frequently implemented as loosely coupled subsystems connected through integration layers. Such separation often introduces synchronization challenges, redundant state representations, and increased operational complexity.

By embedding workflow execution directly into the records management model, the proposed system maintains a single source of truth for both document state and process state. This architectural consolidation reduces impedance mismatch between storage and orchestration layers and enables consistent lifecycle tracking across document-related activities.

The separation between workflow definitions (design-time structures) and workflow instances (runtime state) represents a deliberate design decision aimed at balancing configurability and stability. Versioned definitions allow process evolution without disrupting historical executions, supporting auditability and reproducibility. This distinction is particularly relevant in compliance-sensitive

environments, where historical process logic must remain reconstructible.

From a scalability perspective, the service-oriented backend architecture allows modular evolution of individual components and supports horizontal scaling of stateless services. The introduction of asynchronous messaging for workflow-related events decouples long-running operations from synchronous API interactions, improving system responsiveness and fault tolerance. This event-driven model aligns with contemporary distributed system design practices and enhances resilience under variable load conditions.

The hybrid metadata persistence strategy—combining relational schema definitions with serialized runtime payloads—provides flexibility while preserving structural validation guarantees. However, this approach also introduces trade-offs. Serialized metadata values may limit direct relational querying capabilities and can require additional processing for analytical scenarios. Future extensions may consider selective indexing or projection strategies for frequently queried metadata attributes.

Despite these strengths, certain limitations remain. While the system structurally supports retention policies and disposal types, automated lifecycle enforcement and large-scale disposal workflows require further empirical validation. In addition, comprehensive performance benchmarking under high concurrency and distributed deployment scenarios would provide deeper insight into system behavior at scale.

Overall, the architectural design establishes a technically coherent foundation for enterprise-grade document and records management systems. The integration of classification hierarchies, configurable metadata, workflow orchestration, and compliance-oriented modeling demonstrates that flexibility and regulatory alignment need not be treated as mutually exclusive design goals.

6 Conclusion

This paper presented the architectural design and implementation of a workflow-enabled enterprise document and records management system that integrates compliance-oriented modeling directly into its core domain structure. Rather than treating workflow orchestration and records management as independent subsystems, the proposed solution unifies classification hierarchies, lifecycle attributes, configurable metadata, and versioned workflow execution within a single coherent architecture.

The presented model demonstrates that regulatory alignment, structural consistency, and process flexibility can coexist without introducing architectural fragmentation. By separating workflow definitions from runtime execution state, enforcing schema-driven metadata validation, and embedding retention logic into classification entities, the system establishes a compliance-first foundation while preserving configurability and extensibility.

The architectural approach further illustrates how modern service-oriented design, asynchronous processing, and container-based deployment can support scalable and maintainable enterprise registry solutions. The hybrid metadata persistence strategy and versioned workflow model contribute to both adaptability and auditability, which are essential characteristics of regulated information systems.

Future work will focus on automated lifecycle enforcement mechanisms, extended compliance validation layers, and empirical performance evaluation under large-scale distributed workloads. Additional research may also explore generalization of the proposed model to other regulated domains requiring traceable document-centric workflows.

References

[1] European Parliament and Council. Regulation (eu) no 910/2014 on electronic identification and trust services. https://eur-lex.europa.eu/, 2014. Accessed: 2025-01-06.

[2] European Parliament and Council. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data (gdpr). https://eur-lex.europa.eu/, 2016. Accessed: 2026-02-02.

[3] DLM Forum. Moreq2010 modular requirements for records systems, 2010. URL https://www.moreq.info/. Accessed: 2026-02-14.

[4] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[5] International Organization for Standardization. *ISO 15489-1:2016 Information and Documentation – Records Management – Part 1: Concepts and Principles*. ISO, 2016.

[6] Microsoft. .net documentation. Microsoft Learn, 2026. URL https://learn.microsoft.com/dotnet/. Accessed: 2026-02-14.

[7] Ministerstvo vnútra Slovenskej republiky. Vyhláška č. 628/2002 Z. z., ktorou sa vykonávajú niektoré ustanovenia zákona o archívoch a registratúrach, 2002. Zbierka zákonov SR.

[8] Národná rada Slovenskej republiky. Zákon č. 395/2002 Z. z. o archívoch a registratúrach a o doplnení niektorých zákonov, 2002. Zbierka zákonov SR.

[9] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015.